

# Git Branching & Commit Guide

## 1. الهدف من الـ Git Workflow

إننا شغالين كـ **Frontend + Backend Team** على نفس المشروع، فمحتاجين كلنا نمشي بنفس الطريقة في التعامل مع:

- Branches
- Commits
- Pull Requests
- Code Reviews
- Merging
- Production Releases

الهدف إن كل واحد يعرف يشتغل على الـ **task** بتاعته من غير ما يلخبط شغل باقي التيم، وفي نفس الوقت نبقى عارفين بسهولة:

- مين شغال على إيه؟
- الـ **branch** ده خاص بإيه؟
- الـ **commit** ده عمل إيه؟
- الـ **PR** ده بيحل أنهي **Issue**؟
- التغيير ده رايح للـ **Development** ولا **Production**؟

## 2. يعني إيه **Branch**؟

الـ **Branch** ببساطة هو نسخة منفصلة من الـ **codebase** تقدر تشتغل عليها من غير ما تأثر بشكل مباشر على شغل باقي التيم.

تخيل إن عندنا:

```
Project
|
├── Backend
└── Frontend
```

لو إنت شغال على Login Feature، بدل ما تعدل مباشرة في **dev**، بتعمل branch خاص بالـ Login:

```
dev
|
└─ feature/auth/login
```

وتشتغل عليه.

بعد ما تخلص:

```
feature/auth/login
|
▼
Pull Request
|
▼
dev
```

كده شغلك يدخل للـ development بعد ما يتعمله Review و Testing.

### 3. الـ Main Branches

إحنا عندنا Branches أساسية:

```
production
|
└─ Production Environment

dev
|
└─ Development Environment
```

**production**

ده أهم Branch عندنا.

الـ code الموجود فيه هو الـ code اللي شغال عند الـ end users.

يعني أي حاجة تدخل **production** لازم تكون:

- Tested
- Reviewed
- Stable
- Ready for real users

ممنوع تشتغل مباشرة على **production**.

---

**dev**

ده الـ branch الرئيسي بتاع الـ development.

كل الـ features والـ bug fixes العادية بتتعمل منه.

مثلاً:

```
dev
|
├─ feature/auth/login
├─ feature/courses/filter
├─ bug-fix/cart/total-price
└─ refactor/orders/payment-service
```

بعد ما الـ feature تخلص وتعمل PR ويتم قبولها، تدخل إلى:

**dev**

**خد بالك يا صديقي**

الـ **dev** والـ **production** الاتنين موجودين كـ branches أساسية، والـ **dev** بيكون هو المكان اللي بنجمع فيه شغل الـ development قبل ما يطلع للـ **production**.

---

## 4. Specialized Branches

مش كل Branch بيتعمل من نفس المكان.

عندنا نوعين رئيسيين من الـ work:

### الـ Normal Work

الشغل العادي بيتعمل من:

dev

وده يشمل:

- Features
- Bug Fixes
- Refactoring

### الـ Urgent Production Work

الشغل العاجل الخاص بالـ Production بيتعمل من:

production

وده يشمل:

- الـ Hot Fixes
- الـ Hot Features

## 5. الـ Feature Branches

الـ branch `feature/` بنستخدمه لما نكون بنضيف Feature جديدة.

مثلاً:

الـ Frontend Developer بيضيف صفحة Course Details.

يعمل:

```
git switch dev
git pull origin dev

git switch -c feature/courses/course-details
```

الـ branch هنا معناه:

```
feature
├──
└── courses
    ├──
    └── course-details
```

مثال Backend:

```
feature/courses/get-course-details
```

مثال Frontend:

```
feature/courses/course-details-page
```

## 6. الـ Bug Fix Branches

لو عندك Bug في الـ application، تستخدم:

```
bug-fix/
```

مثلاً:

الـ cart بيحسب الـ total غلط.

تعمل:

```
bug-fix/cart/fix-total-calculation
```

مثال Backend:

```
bug-fix/orders/fix-order-total
```

مثال Frontend:

```
bug-fix/cart/fix-cart-total-display
```

## 7. Refactor Branches

لو هتغير الـ implementation أو تحسن الـ code من غير ما تضيف Feature جديدة، استخدم:

```
refactor/
```

مثلاً:

عايز تحسن طريقة التعامل مع الـ Order Service.

تعمل:

```
refactor/orders/improve-order-service
```

أو:

```
refactor/auth/improve-token-service
```

الـ Refactoring معناه إنك بتحسن الـ code الموجود من غير ما تغير الـ business behavior الأساسي بتاعه.

يعني مثلاً:

الكود بيعمل Login صح، وإنت عايز تخليه أنصف أو أسهل في الصيانة.

ده **Refactor** مش Feature جديدة.

## 8. Hot Fix

الـ `/hot-fix` مختلف عن الـ `/bug-fix`.

### Bug Fix

ده Bug عادي موجود في الـ `development`:

```
dev
├──
└── bug-fix/cart/fix-total
```

### Hot Fix

ده Bug موجود في الـ `Production` ومحتاج يتصلح بسرعة.

مثلاً:

```
production
├──
└── hot-fix/payments/fix-payment-failure
```

مثلاً:

المستخدمين مش قادرين يدفعوا بالـ `PayPal` في `Production`.

نعمل:

```
git switch production
git pull origin production
```

```
git switch -c hot-fix/payments/fix-paypal-error
```

بعد ما نصلح المشكلة ونعمل PR:

```
hot-fix/payments/fix-paypal-error
|
▼
production
```

وبعد كده لازم نرجع نفس الـ fix للـ `dev` عشان المشكلة دي متظهرش تاني لما الـ `dev` يطلع Production.

يعني:

```
hot-fix
|
├───> production
|
└───> dev
```

## 9. Hot Features

الـ `/hot-features` بنستخدمها للـ features العاجلة اللي محتاجة تدخل Production مباشرة.

مثلاً:

```
hot-features/payments/add-new-payment-method
```

الـ branch بيتعمل من:

```
production
```

مش من `dev` .

لكن النوع ده لازم نستخدمه بحذر، لأن أي Feature هتدخل Production مباشرة لازم تكون مستقرة ومختبرة كويس.

## 10. Branch Naming Convention

كل Branch لازم يكون اسمه واضح ويقول هو خاص بإيه.

الـ format:

```
[type-of-work]/[area]/[brief-description]
```

مثلاً:

```
feature/reviews/add-review-photos
```

معناه:

feature



نوع الشغل

reviews



الجزء اللي هنتعامل معاه

add-review-photos



إيه اللي هنعمله

يعني لما حد يشوف اسم الـ branch، يفهم تقريباً إنت شغال على إيه من غير ما يفتح الـ code.

---

## 11. Branch Naming Examples

### Features

```
feature/auth/google-login
feature/courses/add-course-rating
feature/reviews/add-review-photos
feature/cart/apply-coupon
feature/profile/edit-profile
```

## Bug Fixes

```
bug-fix/cart/fix-total-calculation
bug-fix/auth/fix-refresh-token
bug-fix/orders/fix-shipping-calculation
bug-fix/courses/fix-pagination
```

## Refactoring

```
refactor/orders/improve-order-service
refactor/auth/refactor-token-service
refactor/courses/optimize-course-query
```

## Hot Fixes

```
hot-fix/payments/resolve-paypal-error
hot-fix/auth/fix-login-production-error
hot-fix/orders/fix-checkout-failure
```

---

## 12. هل Frontend و Backend يستخدموا نفس الـ Branch؟

الأفضل إن كل Developer يعمل Branch للـ task الذي هو مسؤول عنها.

مثلاً عندنا Feature:

Add Course Reviews

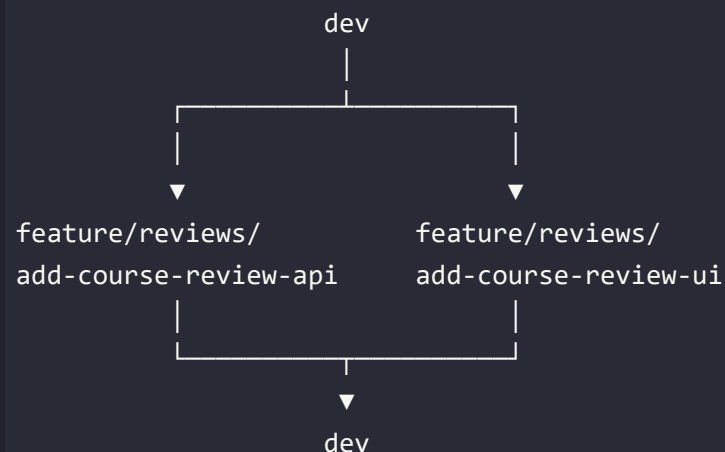
الـ Backend Developer ممكن يعمل:

feature/reviews/add-course-review-api

وال Frontend Developer يعمل:

feature/reviews/add-course-review-ui

ويبقى عندنا:



وده أفضل من إن ال Frontend وال Backend يشتغلوا على نفس ال branch لو كل واحد شغال بشكل مستقل في شغله.

كل واحد يبقى مسؤول عن ال branch بتاعه وال task بتاعته.

## 13. طب لو Frontend محتاج ال Backend API؟

دي نقطة مهمة جدًا.

مثلاً ال Frontend Developer محتاج:

POST /api/reviews

لكن ال Backend لسه بيي عمل ال endpoint.

هنا لازم الفريق يتفق الأول على ال **API Contract**.

- ال Endpoint
- ال HTTP Method
- ال Request
- ال Response
- ال Status Codes
- ال Validation
- ال Authentication

مثلاً:

## Request

```
{
  "courseId": 10,
  "rating": 5,
  "comment": "Great course!"
}
```

## Response

```
{
  "id": 25,
  "rating": 5,
  "comment": "Great course!",
  "userName": "Ahmed"
}
```

ال Frontend يقدر يبدأ شغله بناءً على ال contract ده حتى لو ال Backend لسه بيكمل implementation.

وممكن يستخدم Mock Data مؤقتًا لحد ما ال API تخلص.

الـ Commit هو نقطة محفوظة من الـ changes اللي إنت عملتها.

يعني كل ما تعمل مجموعة تغييرات منطقية، تقدر تحفظها في Commit.

مثلاً إنت خلصت Login API:

```
git add .
git commit -m "feat: add login endpoint"
```

الـ Commit Message لازم توضح إنت غيرت إيه.

مش مهم تكون طويلة، المهم تكون واضحة.

---

## 15. الـ Commit Naming Convention

إحنا هنستخدم **Conventional Commits**.

الـ format:

```
type: description
```

أشهر الـ types اللي هنستخدمها:

```
feat
fix
refactor
docs
test
chore
style
```

---

## 16. feat

بنستخدمها لما نضيف Feature جديدة.

مثلاً:

```
git commit -m "feat: add course review endpoint"
```

مثال Frontend:

```
git commit -m "feat: add course review form"
```

يعني:

أنا ضفت حاجة جديدة للـ application.

---

## 17. fix

بنستخدمها لما نصلح Bug.

مثلاً:

```
git commit -m "fix: correct cart total calculation"
```

أو:

```
git commit -m "fix: handle expired refresh token"
```

يعني:

كان فيه حاجة غلط وأنا صلحتها.

---

## 18. refactor

بنستخدمها لما نحسن الـ code من غير ما نغير الـ behavior.

مثلاً:

```
git commit -m "refactor: simplify order service"
```

يعني الـ functionality نفسها لسه زي ما هي، بس الكود بقى أنضف أو أحسن.

---

## 19. docs

بنستخدمها لما نعدل الـ Documentation.

مثلاً:

```
git commit -m "docs: update authentication guide"
```

## 20. test

بنستخدمها لما نضيف أو نعدل الـ Tests.

مثلاً:

```
git commit -m "test: add login handler tests"
```

## 21. chore

بنستخدمها في الـ changes اللي مش Feature ولا Bug Fix، زي:

- تحديث Packages
- تعديل Docker

مثلاً:

```
git commit -m "chore: update project dependencies"
```

## 22. style

بنستخدمها لتغييرات الـ formatting أو style اللي مش بتغير الـ behavior.

مثلاً:

```
git commit -m "style: format authentication services"
```

## 23. Commit Message Rules

اكتب الـ commit كأنه بيجاوب على السؤال:

What did I change?

يعني:

إنت عملت إيه؟

بدل ما تكتب:

```
git commit -m "changes"
```

اكتب:

```
git commit -m "feat: add course enrollment endpoint"
```

وبدل:

```
git commit -m "fix"
```

اكتب:

```
git commit -m "fix: prevent duplicate course enrollment"
```

## 24. خلي الـ Commits صغيرة ومنطقية

متعملش Commit واحد فيه 10 حاجات مختلفة.

Bad:

```
feat: update everything
```

وجواه:

```
Login
Register
Courses
Cart
Database
UI
```

ده بيصعب جدًا الـ Code Review وبيصعب علينا نعرف إيه اللي حصل.

الأفضل:

```
feat: add login endpoint
feat: add registration validation
feat: add course filtering
fix: correct cart total
```

## 25. الـ Workflow كامل للـ Developer

لما تاخذ Task جديدة، امشي بالخطوات دي.

### Step 1 — Update dev

أول حاجة اتأكد إن عندك آخر نسخة من `dev` :

```
git switch dev
git pull origin dev
```

### Step 2 — Create Branch

مثلاً عندك Task:

Add Course Rating

اعمل:

```
git switch -c feature/courses/add-course-rating
```

كده بقيت شغال على branch خاص بالـ task بتاعتك.

### Step 3 — Work

ابدأ تنفيذ الـ task.

مثلاً:

- Create Entity
- Create Repository
- Create Handler
- Create Endpoint
- Add Validation
- Add Tests

مش شرط كل خطوة من دول تكون Commit لوحدها، المهم إن الـ commits تكون منطقية.

---

## Step 4 — Commit

لما تخلص جزء منطقي من الشغل:

```
git add .
git commit -m "feat: add course rating entity"
```

ثم:

```
git add .
git commit -m "feat: add course rating endpoint"
```

ثم:

```
git add .
git commit -m "test: add course rating tests"
```

---

## 26. Push الـ Branch

بعد ما تخلص:

```
git push -u origin feature/courses/add-course-rating
```

كده الـ branch بتاعك بقى موجود على الـ Remote Repository، والـ team يقدر يشوفه.

## 27. Pull Request

بعد الـ Push تعمل:

Pull Request

من:

feature/courses/add-course-rating

إلى:

dev

يعني:

feature/courses/add-course-rating  
|  
| Pull Request  
▼  
dev

الـ PR معناه:

أنا خلصت الـ task بتاعتي، وعايذكُم تراجعوا التغييرات بتاعتي وبعد الموافقة تدخل في **dev**.

## 28. الـ Pull Request مش مجرد Merge

الـ PR مش مجرد زر الـ Merge.

الـ PR هو المكان اللي الـ team بيتأكد فيه إن الـ change:

- بيحل الـ task المطلوبة.

- مفيهوش Bugs واضحة.
  - مفيهوش Breaking Changes غير متفق عليها.
  - متوافق مع Coding Standards.
  - ال Tests موجودة لو مطلوبة.
  - ال Frontend وال Backend متوافقين.
- يعني ال PR جزء أساسي من ال Code Review.

## 29. قبل ما تعمل PR

قبل ما تفتح ال PR، اتأكد إن ال branch بتاعك محدث بآخر changes موجودة في `dev`.

```
git switch dev
git pull origin dev
```

ثم ارجع للـ branch بتاعك:

```
git switch feature/courses/add-course-rating
```

وبعدھا حدّث ال branch بتاعك من `dev`.

ممکن باستخدام `merge`:

```
git merge dev
```

أو باستخدام `rebase` لو الفريق متفق إنه يستخدمه:

```
git rebase dev
```

لو حصل conflicts، لازم تحلها وتختبر المشروع كويس قبل فتح ال PR.

## 30. PR Target

الشغل العادي:

```
feature/*   —————> dev
bug-fix/*   —————> dev
refactor/*  —————> dev
```

الشغل العاجل:

```
hot-fix/*   —————> production
hot-features/* —————> production
```

وبعد الـ Hot Fix:

```
hot-fix/*
|
|————> production
|
|————> dev
```

عشان الـ fix يفضل موجود في الـ development وميرجعش يظهر تاني لما نعمل Release جديد.

## 31. مثال كامل — Backend

Task:

Add endpoint for creating course reviews.

### Create Branch

```
git switch dev
git pull origin dev

git switch -c feature/reviews/add-course-review-api
```

## Commits

```
git add .  
git commit -m "feat: add course review entity"
```

```
git add .  
git commit -m "feat: add create course review handler"
```

```
git add .  
git commit -m "feat: add create course review endpoint"
```

```
git add .  
git commit -m "test: add course review tests"
```

## Push

```
git push -u origin feature/reviews/add-course-review-api
```

## PR

```
feature/reviews/add-course-review-api  
|  
▼  
dev
```

---

## 32. مثال — Frontend

Task:

Create UI for course reviews.

## Create Branch

```
git switch dev
git pull origin dev

git switch -c feature/reviews/add-course-review-ui
```

## Commits

```
git add .
git commit -m "feat: add course review component"
```

```
git add .
git commit -m "feat: integrate course review API"
```

```
git add .
git commit -m "fix: handle review submission error"
```

## Push

```
git push -u origin feature/reviews/add-course-review-ui
```

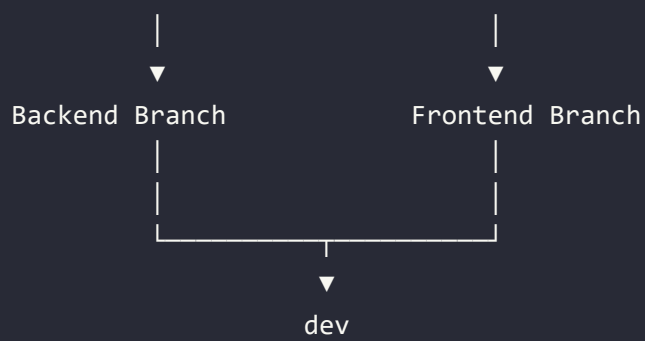
## PR

```
feature/reviews/add-course-review-ui
|
▼
dev
```

## 33. الـ Frontend + Backend Feature

لو Feature محتاجة Frontend و Backend:





مثلاً:

```
feature/auth/google-login-api  
feature/auth/google-login-ui
```

الـ Backend يعمل الـ API Contract.

الـ Frontend يشتغل على الـ UI بناءً على الـ contract.

بعد كده الاتنين يدخلوا **dev**.

## 34. مهم جداً: متشتغلش على **dev** مباشرة

ممنوع تعمل:

```
git switch dev  
  
# modify files  
  
git add .  
git commit -m "my changes"  
git push
```

بدل كده:

```
git switch dev
git pull origin dev

git switch -c feature/...
```

وبعدین اشتغل على الـ feature branch بتاعك.

---

## 35. متعملش Branch من Branch شخص تاني من غير سبب

مثلاً عندك:

```
feature/auth/login
```

ومتعملش:

```
feature/auth/login-ui
```

من الـ branch الـ Backend لمجرد إنك محتاج الـ Login.

الأفضل إن كل واحد يبدأ من:

```
dev
```

إلا لو فيه Dependency واضحة ومؤقتة بين الشغلين.

يعني لو مفيش سبب قوي، كل Task تبدأ من `dev`.

---

## 36. لو الـ Frontend محتاج Backend ولسه مش جاهز

متستناش بالضرورة.

اتفقوا الأول على:

Endpoint  
HTTP Method  
Request  
Response  
Status Codes  
Validation Errors  
Authentication

مثلاً:

POST /api/courses/{courseId}/reviews

Request:

```
{
  "rating": 5,
  "comment": "Excellent course"
}
```

Response:

```
{
  "id": 15,
  "rating": 5,
  "comment": "Excellent course",
  "userName": "Ahmed"
}
```

الـ Frontend يقدر يستخدم Mock Response أثناء تطوير الـ UI لحد ما الـ Backend يخلص.

المهم إن الاتنين يكونوا متفقين على الـ API Contract من الأول.

---

## 37. متخاطش Features مختلفة في نفس Branch

Bad:

feature/courses/all-changes

وتحط جواه:

Course Details

Cart

Authentication

Profile

Reviews

الأفضل:

feature/courses/course-details

feature/cart/add-coupon

feature/auth/login

feature/profile/edit-profile

feature/reviews/add-review

ده بيخلي الـ PR:

- أصغر
- أسهل في الـ Review
- أسهل في الـ Testing
- أسهل في الـ Revert
- أقل عرضة للـ Conflicts

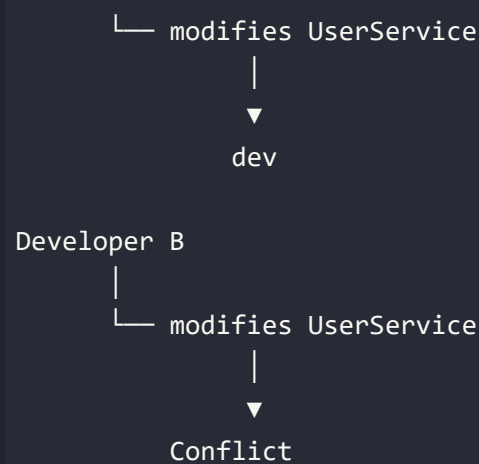
## 38. التعامل مع Merge Conflicts

الـ Conflict بيحصل لما شخصين يعدلوا نفس الجزء من نفس الملف بشكل متعارض.

مثلاً:

Developer A

|



لو حصل Conflict:

1. متعلمش Panic.

2. اقرأ التغييرات الموجودة.

3. افهم شغل الطرفين.

4. شوف إيه التغيير الصح.

5. ادمج التغييرات لو الاتنين محتاجين بعض.

6. Run the project.

7. Run tests.

8. بعد ما تتأكد إن كل حاجة تمام، اعمل Commit للحل.

مثلاً:

```
git add .
git commit -m "fix: resolve merge conflicts"
```

## 39. Pull Request Checklist

قبل ما تعمل PR اسأل نفسك:

- [ ] dev؟ مبني من آخر branch هل الـ
- [ ] صحيح؟ branch هل اسم الـ

```
[ ] بيعمل المطلوب؟ code هل ال
[ ] feature؟ هل جربت ال
[ ] لو مطلوبة؟ tests هل عملت
[ ] Debug code؟ هل في
[ ] غير ضرورية؟ Console.WriteLine / logs هل في
[ ] Frontend؟ متوافق مع API contract هل ال
[ ] موجود؟ Error handling هل ال
[ ] واحدة واضحة؟ Task فيه PR هل ال
```

## 40. PR Description

ال PR لازم يكون واضح.

مثلاً:

### ## What

Added course review creation API.

### ## Changes

- Added CourseReview entity
- Added create review command
- Added validation
- Added API endpoint
- Added unit tests

### ## API

POST /api/courses/{courseId}/reviews

### ## Testing

- Tested valid review
- Tested invalid rating

- Tested unauthorized request
- Tested duplicate review

الهدف إن أي حد في التيم يفتح ال PR يفهم بسرعة:

- إيه اللي اتعمل؟
- إيه اللي اتغير؟
- إزاي يختبره؟

---

## 41. Rules مهمة للفريق

### Rule 1

Never push directly to production.

---

### Rule 2

Never push directly to dev unless the team explicitly agrees.

الشغل الطبيعي يكون من خلال:

Branch → Pull Request → Review → Merge

---

### Rule 3

كل Task لها Branch واضح.

---

### Rule 4

## Rule 5

كل Commit لازم يكون مفهوم.

## Rule 6

متعملش Commit برسالة زي:

```
update  
changes  
fix  
test  
done  
final  
final-final
```

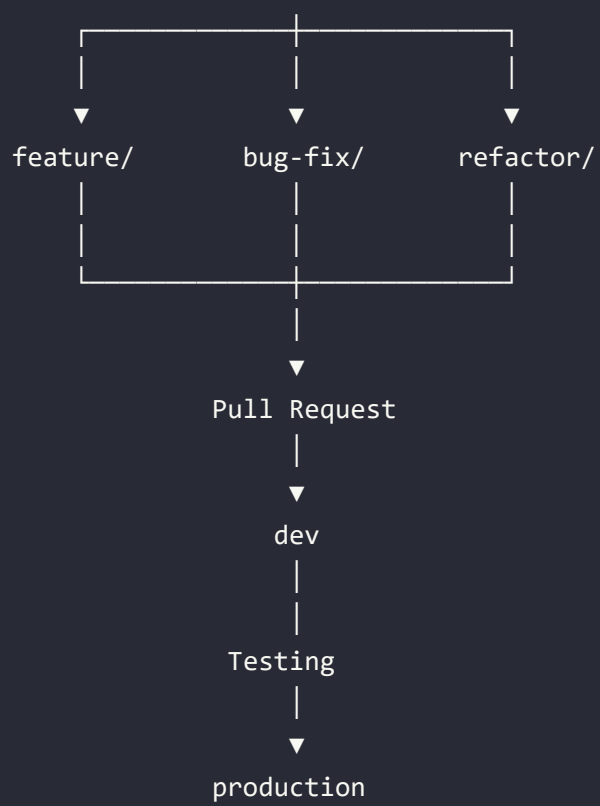
استخدم:

```
feat: add course enrollment endpoint  
fix: prevent duplicate enrollment  
refactor: simplify enrollment service  
test: add enrollment handler tests
```

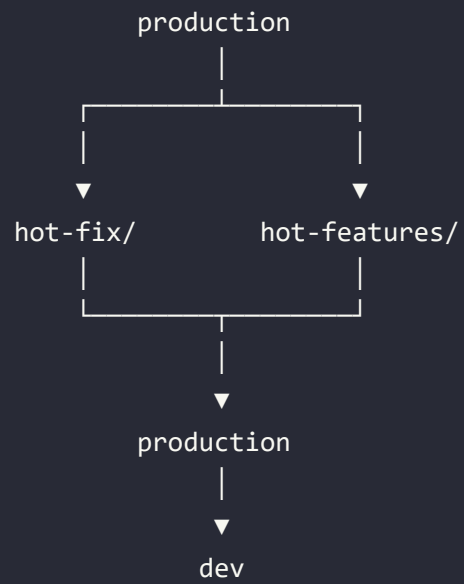
## 42. الشكل النهائي للـ Workflow

الشغل العادي:

```
dev  
|
```



الشغل العاجل:



## 43. Quick Reference

| Work                      | Branch        | Created From | Merge To         |
|---------------------------|---------------|--------------|------------------|
| New Feature               | feature/      | dev          | dev              |
| Bug Fix                   | bug-fix/      | dev          | dev              |
| Refactoring               | refactor/     | dev          | dev              |
| Urgent Production Bug     | hot-fix/      | production   | production + dev |
| Urgent Production Feature | hot-features/ | production   | production + dev |

---

## Quick Developer Workflow

احفظ الخطوات دي:

```
# 1. Update dev
git switch dev
git pull origin dev

# 2. Create your branch
git switch -c feature/courses/add-course-rating

# 3. Work...

# 4. Commit your changes
git add .
git commit -m "feat: add course rating endpoint"

# 5. Push
git push -u origin feature/courses/add-course-rating

# 6. Open Pull Request
# feature/... → dev
```

```
# 7. Review
# 8. Fix requested changes
# 9. Merge
# 10. Delete the branch after merge
```

## 44. أهم فكرة لازم الفريق يفهمها

إحنا مش بنستخدم Git بس عشان نحفظ الـ code.

إحنا بنستخدمه عشان ننظم طريقة تعاون الفريق.

كل Branch لازم يجاوب:

أنا شغال على إيه؟

كل Commit لازم يجاوب:

أنا غيرت إيه؟

كل Pull Request لازم يجاوب:

ليه التغيير ده موجود؟ وهل هو جاهز يدخل الـ project؟

وبالتالي الـ workflow الطبيعي للفريق يكون:

```
Task
↓
Create Branch
↓
Develop
↓
Commit
↓
Push
↓
Pull Request
```

↓  
Code Review  
↓  
Testing  
↓  
Merge to dev  
↓  
Integration Testing  
↓  
Release  
↓  
production

الفكرة الأساسية:

اشتغل على Branch خاص بالـ task بتاعتك، اعمل Commits واضحة، افتح Pull Request، خلي حد يراجع شغلك، وبعد الموافقة يدخل التغيير للـ **dev**.